Throughput Evaluation

The final data plane performance metric evaluated was throughput, which measures the effective data transfer rate over a sustained period. This metric is crucial for assessing the framework's capacity to handle bulk data transfers, reflecting the combined efficiency of the routing logic, forwarding process, and the underlying wireless medium under heavy load.

To achieve a measurement that accurately reflects the performance of our user-space implementation, a custom data transfer mechanism was designed and built directly into the OLSR framework. This bespoke approach was chosen over standard OS-level tools (e.g., iperf) for two key reasons. First, standard utilities would bypass our application's user-space forwarding logic, thereby failing to test the complete system. Second, a custom mechanism provides the fine-grained control necessary for detailed instrumentation and analysis.

The design of this custom data transfer mechanism was conceptually based on the principles of the Trivial File Transfer Protocol (TFTP) [81]. Similar to TFTP, our implementation operates over UDP and relies on segmenting a file into fixed-size chunks for transmission. This bespoke approach, tailored specifically for this evaluation, allows for a direct measurement of the data plane's performance by ensuring that all packet forwarding is handled exclusively by the user-space OLSR framework.

The experimental procedure was as follows:

1. Test Orchestration and High-Load Conditions: The test was initiated from the terminal of a source node using the `test_runner.py` script. Unlike the previous PDR and delay tests which used a 0.5-second interval, this test was designed to impose a maximum load on the network. Chunks were

transmitted with a minimal 1 ms delay, creating a continuous, high-rate stream of traffic that stressed the processing capacity of each node.

Example Command:

```
python3 test_runner.py throughput 192.168.10.2 192.168.10.1 1
```

2. Data Transfer Mechanism:

   - File Segmentation: At the source node, a 1 MB file was segmented into 1024 chunks, each chunk is 1024 bytes in size.
   - Packet Encapsulation: Each chunk was encapsulated in a JSON-based data packet containing essential metadata: a unique `file_id`, the `chunk_id`, and the `total_chunks` count for reassembly.
   - User-Space Forwarding: Packets were sent sequentially. Each node in the network, upon receiving a data packet, used its OLSR-computed routing table to determine the next hop and forward the packet accordingly.

3. Throughput Calculation: The destination node collected the incoming chunks. The transfer was considered complete when the count of received unique chunks equaled the `total_chunks` value. The throughput was then calculated using the following formula, with the result scaled to Megabits per second (Mbps):

   $$Throughput(Mbps) = (Total\ File\ Size\ in\ Bytes \times 8) \div (Total\ Transfer\ Duration\ \ in\ sec\ \times 1024\ \times 1024)$$

4. Repetitive Measurement: For each scenario (1-hop, 2-hop, and 3-hop), the 1 MB file transfer was repeated five times to ensure the consistency of the results. The average throughput results for each scenario are summarized in the tables below.

Table 1: Throughput Measurement Results for 1-Hop Scenario

# Trial	Path (Source -> Destination)	Latency (Sec)	Throughput (Mbps)
1	Node 2 -> Node 1	2.0144	3.97
2	Node 2 -> Node 1	2.0456	3.91
3	Node 2 -> Node 1	2.0271	3.95
4	Node 2 -> Node 1	2.0652	3.87
5	Node 2 -> Node 1	2.0302	3.94
Average		**2.0365**	**3.92**

Table.2: Throughput Measurement Results for 2-Hop Scenario

# Trial	Path (Source ->Destination)	Latency (Sec)	Throughput (Mbps)
1	Node 3 -> Node 2 -> Node 1	2.5681	3.12
2	Node 3 -> Node 2 -> Node 1	2.5472	3.14
3	Node 3 -> Node 2 -> Node 1	2.5337	3.16
4	Node 3 -> Node 2 -> Node 1	2.5531	3.13
5	Node 3 -> Node 2 -> Node 1	2.5873	3.09
Average		**2.578**	**3.128**

Table 3: Throughput Measurement Results for 3-Hop Scenario

# Trial	Path (Source -> Destination)	Latency (Sec)	Throughput (Mbps)
1	Node 5 -> Node 3 -> Node 2 -> Node 1	3.7738	2.12
2	Node 5 -> Node 3 -> Node 2 -> Node 1	3.784	2.11
3	Node 5 -> Node 3 -> Node 2 -> Node 1	3.9322	2.03
4	Node 5 -> Node 3 -> Node 2 -> Node 1	3.7302	2.14
5	Node 5 -> Node 3 -> Node 2 -> Node 1	3.7462	2.13
Average		**3.7932**	**2.106**

The results demonstrate a clear and predictable degradation in throughput as the number of hops increases. The 1-hop path established a baseline performance of approximately 3.92 Mbps. As expected in multi-hop wireless networks, introducing intermediate forwarding nodes resulted in a measured decrease in this rate. This performance degradation can be attributed to two primary factors operating at different layers of the network stack:

- Processing and Queuing Delay (Upper Layers): Each intermediate node must receive a packet, process its header, look up the next hop in its user-space routing table, and then re-queue the packet for transmission. This

computational overhead, although minimal on a per-packet basis, accumulates significantly under a high-rate stream, creating processing bottlenecks and increasing the overall transfer time.

- Medium Contention and Collisions (Lower Layers): In a shared wireless medium, each forwarding action consumes channel resources. A 2-hop transfer requires the medium to be used twice for each packet (source-to-intermediate, intermediate-to-destination). This increases medium contention and the probability of packet collisions, which reduces effective throughput.

The 2-hop path averaged 3.13 Mbps, while the 3-hop path saw a further reduction to 2.11 Mbps, see Figure 1. This trend confirms that both processing overhead and medium contention are significant factors. The consistency of the results across the five trials, even under

this high-load scenario, validates the stability of the framework's forwarding engine. The achieved throughput values are robust for a Python-based user-space implementation on resource-constrained hardware.

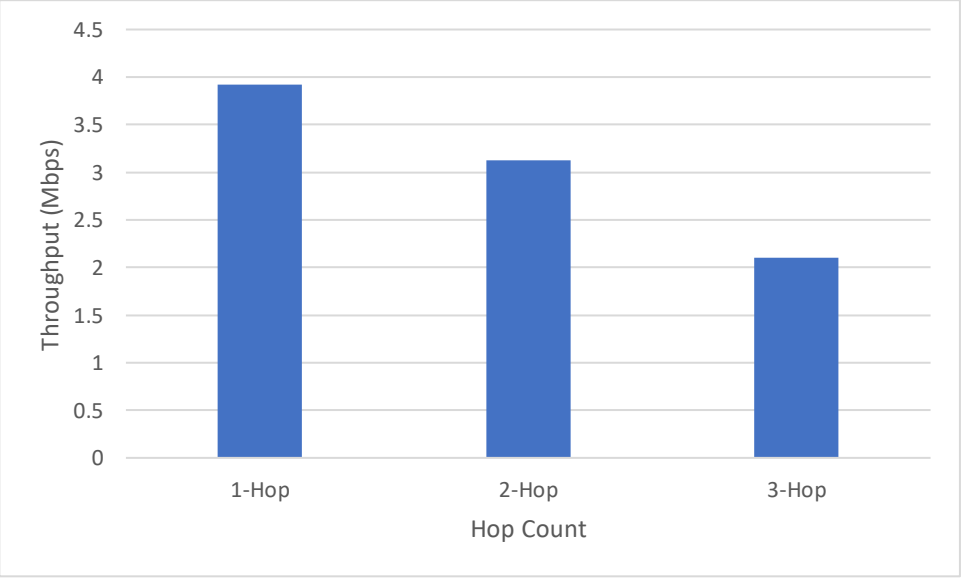

Figure.2: Impact of Hop Count on Average Data Throughput